hashlock.

Security Audit

DrDoge (Meme Coin)

June 2025

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	20
Conclusion	21
Our Methodology	22
Disclaimers	24
About Hashlock	25

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

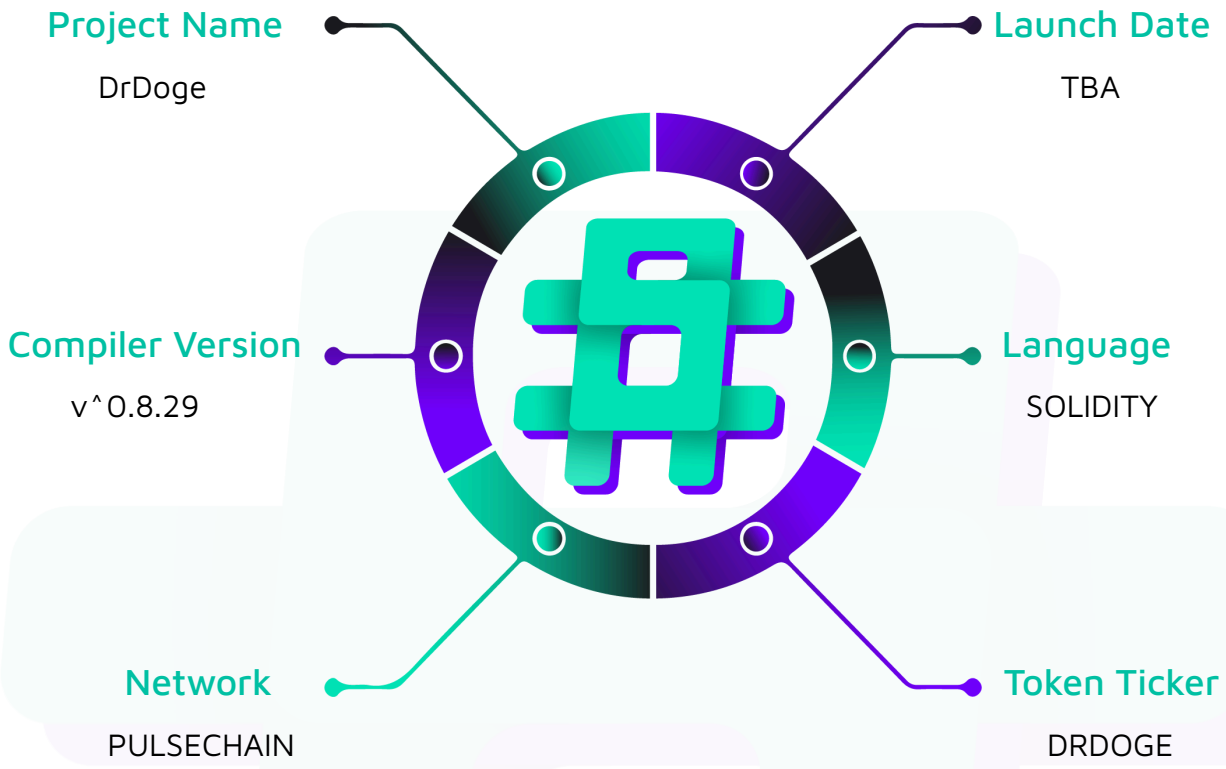
The DrDoge team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

DrDoge is a community-driven meme coin and PRC-20 token deployed on PulseChain, designed to incentivize meaningful scientific contributions via blockchain technology. Participants provide GPU and CPU computational resources to support distributed biomedical research initiatives. Verified scientific contributions are rewarded with DrDoge tokens through a transparent and traceable system, establishing a direct link between decentralized meme coin incentives and measurable scientific outcomes.

Project Name: DrDoge
Project Type: Meme Coin
Compiler Version: ^0.8.29
Logo:

Visualised Context:



Project Visuals:

Audit Scope

We at Hashlock audited the solidity code within the DrDoge project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	DrDoge Smart Contracts
Platform	PulseChain / Solidity
Audit Date	June, 2025
Contract 1	DrDoge_Token.sol
Contract 2	DrDoge_LP_Lock.sol
Audited GitHub Commit Hash	8f98b26a14df6c756745cc20dd37a2f7e91947b8
Fix Review GitHub Commit Hash	53b1884612c48b0e4d2ca383e1cf7297b84f908d
Deployed Contract 1 Address	0x9d95DFBB1aBb8F57abc20FDE4F786BB1Db52ad71
Deployed Contract 2 Address	0x5837f87ACacaF6204FFf31883b20b6D08DFbb753

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerability

2 Low severity vulnerabilities

2 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
DrDoge_Token.sol - Mint initial liquidity pool starting tokens (one-time) - Mint one-time janitor tokens (one-time) - Mint monthly CureCoin & FLDC claim tokens (after 4-month delay, up to 6 months) - Mint monthly liquidity pool starting sacrifice rewards and success lock tokens (up to 24 months each) - Mint yearly janitor allocation (unlimited years, first mint only after 1 year) - Mint monthly community sacrifice lock tokens (up to 24 months) - Mint monthly community buyback lock tokens (up to 24 months) - Mint yearly staking rewards (up to 3 years) - Mint yearly science reward tokens (unlimited, tiered schedule for first 21 years) - Mint a one-time liquidity pool start success lock allocation - Standard ERC20 token functionalities	**Contract achieves this functionality.**
DrDoge_Lock.sol - The locking contract contains no functions and consists solely of comments. As a result, any tokens, including LP tokens, sent to this contract are permanently inaccessible and can be considered truly "locked." From a technical standpoint, the contract is secure: no one can withdraw funds from it under any circumstances.	**The contract has no functionality.**

Code Quality

This audit scope involves the smart contracts of the DrDoge project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the DrDoge project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] DrDoge#mintCureCoinAndFLDCClaim - Critical Mathematical Flaw Leads to Incorrect Monthly Slot Availability

Description

The `mintCureCoinAndFLDCClaim` function is designed to allow the `JustAJanitor` to mint tokens designated for `CureCoin` and `FLDC` claims on a monthly basis, after an initial 4-month delay(`CLAIM_START_DELAY`) and for a total of 6 claim periods (`cureCoinAndFLDCClaimAmount < 6`)

The identified vulnerability lies in the specific line of code responsible for calculating `availableSlots`:

```
uint256 availableSlots = monthsElapsed >= 6 ? monthsElapsed - 6 + 1 : 0;
```

This calculation is flawed due to an incorrect offset. The logic implies that claimable slots only start accumulating or become recognizable once monthsElapsed reaches 6. However, the `CLAIM_START_DELAY` is set to 4 * `BLOCKS_PER_MONTH`, meaning the claiming period should effectively begin after 4 months have passed since the `startBlock`

Vulnerability Details

The core of the issue stems from an inconsistency and incorrect assumption:

1. Maximum Allowed Claims: The given line correctly establishes that there are a total of 6 distinct claim periods.

```
require(cureCoinAndFLDCClaimCount < 6, "Claim minting period completed");
```

2. Claim Start Delay: The constant `CLAIM_START_DELAY` (equal to 4 months of blocks) correctly mandates that claims cannot begin before 4 months have passed from `startBlock`. This is enforced by the given line:

```
require(block.number >= startBlock + CLAIM_START_DELAY, "Claiming not started yet");
```

3. Incorrect Offset in Slot Calculation: The calculation ` monthsElapsed - 6 + 1 ` uses an offset of `6`. This effectively means that even if 4 or 5 months have passed (satisfying the `CLAIM_START_DELAY`), the `availableSlots` will compute to `0`, preventing any minting for these eligible periods. Only when `monthsElapsed` is 6 will `availableSlots` become 1 (i.e, 6 - 6 + 1)

This creates a discrepancy where the first two eligible claim months (month 4 and month 5 post-deployment) are not correctly accounted for by the availableSlots logic. The intention should be that after 4 months, the first slot becomes available. The calculation should therefore use an offset reflecting this 4-month start delay.

Impact

This mathematical error will directly disrupt the intended token distribution schedule. Leads to token distribution disruption as the first and second months after the 4-month will be effectively skipped or delayed.

Recommendation

To fix this error, update the `availableSlots` logic to:

```
// Define constants at contract or function level for clarity

uint256 constant CLAIM_START_AFTER_MONTHS = 4;  // Claiming can begin after 4 full months have passed

uint256 constant MAX_NUMBER_OF_CLAIMS = 6;          // Maximum number of distinct claim periods allowed




// ... inside mintCureCoinAndFLDCClaim() ...

require(cureCoinAndFLDCClaimCount < MAX_NUMBER_OF_CLAIMS, "Claim minting period completed");
```

```
// monthsElapsed counts full months passed since startBlock
uint256 monthsElapsed = (block.number - startBlock) / BLOCKS_PER_MONTH;



uint256 calculatedAvailableSlots = 0;
if (monthsElapsed >= CLAIM_START_AFTER_MONTHS) {
    // Number of claimable periods that have occurred since claim period started.
    // E.g., if monthsElapsed is 4 (meaning 5th month technically, or end of 4th month),
    // and CLAIM_START_AFTER_MONTHS is 4, then (4 - 4 + 1) = 1 slot.
    // If monthsElapsed is 5, then (5 - 4 + 1) = 2 slots.
    calculatedAvailableSlots = monthsElapsed - CLAIM_START_AFTER_MONTHS + 1;

    // Ensure available slots do not exceed the total number of claims possible.
    if (calculatedAvailableSlots > MAX_NUMBER_OF_CLAIMS) {
        calculatedAvailableSlots = MAX_NUMBER_OF_CLAIMS;
    }
}

require(calculatedAvailableSlots > cureCoinAndFLDCClaimCount, "No monthly slot available");
// uint256 toMint = calculatedAvailableSlots - cureCoinAndFLDCClaimCount;
// ... rest of minting logic
```

Status

Resolved

Low

[L-01] DrDoge#mintScienceReward - Premature Science Reward Minting at Contract Inception

Description

The `mintScienceReward` function is intended to allow the `JustAJanitorto` mint yearly science rewards. The logic uses an internal helper `_yearsSinceStart` to determine the number of "slots" or years eligible for reward minting. `_yearsSinceStart` is calculated as ((block.number - `startBlock`) / `BLOCKS_PER_YEAR`) + 1;.

The vulnerability arises because if `mintScienceReward` is called in the `same` *block* as the contract's deployment (`block.number == startBlock`), then (block.number - startBlock) will be `0`. Consequently, `_yearsSinceStart` will return `((0) / BLOCKS_PER_YEAR) + 1 = 1`.

The subsequent check `require(slots > scienceRewardCount, "No yearly slot available");`(where `scienceRewardCount` is initially `0`) will pass (`1 > 0`), allowing the "Year 1" science reward to be minted immediately upon deployment, without an actual year having passed. This contradicts the common understanding and implementation of other yearly minting functions in the contract (e.g., mintJustYearlyJanitor, mintStakingReward) which explicitly require `block.number >= startBlock + BLOCKS_PER_YEAR`.

Recommendation

To align `mintScienceReward` with other yearly mint functions and prevent premature minting, an explicit block-based delay check should be added, similar to those in other yearly functions.

```
function mintScienceReward() external {

    require(msg.sender == JustAJanitor, "Only JustAJanitor can mint");

    // Add this explicit check to ensure at least one year has passed

     require(block.number >= startBlock + BLOCKS_PER_YEAR, "First year for science reward not yet complete");
```

```
    uint256 slots = _yearsSinceStart();
    // The definition of 'slots' might need re-evaluation if the above check is added.
     // If _yearsSinceStart() is intended to mean "which year's reward is this for", then +1 might be okay.
    // If it means "how many yearly periods have *fully passed*", the +1 is problematic.
     // Alternative calculation for 'slots' or number of eligible mints, after ensuring one year passed:
     // uint256 yearsPassed = (block.number - startBlock) / BLOCKS_PER_YEAR; // Number of full years completed
      // require(yearsPassed > scienceRewardCount, "No yearly slot available or already minted");



     require(slots > scienceRewardCount, "No yearly slot available"); // This line might need adjustment based on refined slot calculation


    // ... rest of the function logic ...
      // Consider how 'slots' interacts with 'scienceRewardCount' if batching is also implemented (see next finding).
    // If batching, 'slots' could represent the *latest* year eligible for minting.


    scienceRewardCount++; // This only handles one year at a time.
    uint256 yearIndex = scienceRewardCount > 21 ? 21 : scienceRewardCount;
    uint256 reward = scienceRewardByYear[yearIndex] * 10**decimals();
    _mint(RECIPIENT_SCIENCE_REWARD, reward);
                emit    MintScienceReward(msg.sender,    RECIPIENT_SCIENCE_REWARD,    reward, scienceRewardCount, block.number, block.timestamp);
```

```
}
```

Further, consider adjusting `_yearsSinceStart` to (`block.number - startBlock`) / `BLOCKS_PER_YEAR` if it's meant to represent completed periods, and handle the "current eligible year" logic within the mint function itself.

Status

Acknowledged

[L-02] DrDoge#mintScienceReward - Inefficient Science Reward Catch-up Mechanism (Single Mint Per Transaction)

Description

The `mintScienceReward` function currently only allows the `JustAJanitor` to mint the reward for a single year per transaction. This is evident as `scienceRewardCount` is incremented by only one (`scienceRewardCount++;`), and a single year's reward is minted. Most other periodic minting functions within this contract (e.g., `mintLiquidityPoolStartingSacrificeReward`, mintJustYearlyJanitor) implement a "catch-up" mechanism, allowing multiple missed periods (up to `MAX_SLOTS_PER_TX`) to be minted in a single transaction. The `mintScienceReward` function lacks this efficiency.

This leads to Increased Gas Costs and Operational Complexity and Risk of Missed Mints along with Potential Delays in Reward

Recommendation

Implement a batch catch-up mechanism for `mintScienceReward()` similar to that used in other periodic minting functions within the contract. This involves calculating the number of eligible but unminted yearly slots and iterating to mint them, respecting `MAX_SLOTS_PER_TX`.

Status

Resolved

QA

[Q-01] DrDoge - Use of Floating Pragma Version

Description

The contract specifies the Solidity compiler version using a floating pragma: `pragma solidity ^0.8.29;`. The caret (`^`) symbol allows the contract to be compiled with any compiler version from `0.8.29` up to (but not including) `0.9.0`. While convenient during development, this practice is not recommended for production deployments.

Recommendation

For production deployments, it is strongly recommended to use a fixed pragma version. This ensures that the contract is always compiled with the exact same compiler version that was used for final testing, auditing, and verification.

Status

Resolved

[Q-02] DrDoge - Redundant Contract Inheritance (ERC20 and ERC20Burnable)

Description

The DrDoge contract declaration includes inheritance from both ERC20 and ERC20Burnable: contract DrDoge is ERC20, ERC20Burnable {. The ERC20Burnable contract from OpenZeppelin (and most standard implementations) already inherits from ERC20. Therefore, explicitly inheriting ERC20 again is redundant.

Recommendation

Simplify the inheritance structure by removing the explicit inheritance from ERC20, as ERC20Burnable already provides it.

Status

Resolved

Centralisation

The DrDoge project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the DrDoge project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au